Лабораторна робота №1

Розгортання середовища та порівняльний аналіз мовних моделей

Дисципліна: Агенти (обчислювальні методи)
Варіант: 18 (за таблицями 1–10 — варіант 8)
Виконав: студент групи AI-244 Русаков Яків
Перевірив: ас. Баранов О.М.
Одеса — 2026

Конфігурація

- ОС / GPU: Google Colab (Linux, віртуальна машина з GPU NVIDIA T4); CPU/RAM окремо не знімалися
- Ollama: версія 0.40.16, встановлена скриптом `install.sh` у Colab станом на 27.10.2026; моделі: `llama3.2:3b`, `qwen3:4b`
- Хмарний провайдер: Groq (`https://api.groq.com/openai/v1`), модель `openai/gpt-oss-120b`
- Python: 3.13.18 (середовище Google Colab)

Завдання 1. Міркувальна vs інструктивна модель (`task1_reasoning.py`, `task1_results.json`)

Дані — з реального запуску в Google Colab (GPU T4, Ollama); журнал — `lab1.log`, сирі дані — `task1_results.json`. Першому запиту до моделі передує її завантаження, тому час першого запиту може бути більшим.

Фактичне питання («Столиця України? Відповідай одним словом.»)

Показник	qwen3:4b	llama3.2:3b
response	«Киев»	«Київ.»
eval_count	1329	5
Час, с	21.6	0.08
Токенів/с	61.9	105.2

Задача на міркування (середнє з 5 запусків; правильна відповідь — 5 грн)

Показник	qwen3:4b	llama3.2:3b
Правильних із 5	5	0
eval_count	1183	177
Час, с	20.1	2.4

Окремі запуски. qwen3:4b: eval_count 1032 / 1224 / 1274 / 975 / 1409, час 17.2 / 20.6 / 21.9 / 16.6 / 24.0 с; у всіх п’яти відповідь «5 грн» через систему рівнянь і перевірку (у двох запусках — 1-му й 4-му — розв’язання написано російською). llama3.2:3b: відповіді 10, 100, 90, 110 і 50 грн — жодної правильної; усі п’ять написані російською (місцями суржиком), хоча питання українською. Прапорець `auto_correct` у журналі хибний для всіх запусків qwen3:4b: автоперевірка шукала «5 грн», а модель записує відповідь як `\boxed{5}`. Тому кількість правильних визначено вручну, прочитавши відповіді.

Спостереження. (1) На простому питанні qwen3:4b згенерував 1329 токенів проти 5 в llama3.2:3b за ту саму відповідь про Київ, причому видав її російською («Киев»). Швидкість генерації близька (62 і 105 токенів/с), а час на задачі міркування різниться в ~8 разів (20.1 проти 2.4 с) — через обсяг роботи. (2) На задачі з пасткою (інтуїтивно «10 грн») перевагу міркувальної моделі видно найкраще: 5/5 проти 0/5. (3) Результати різних запусків відрізняються (у попередньому прогоні qwen3:4b на ті самі питання витратив 723 токени й відповів «Київ»): відповіді моделі недетерміновані.

Відповіді чат-бота на 5 питань (стисло)

1. *Куди пішли токени?* У прихований ланцюжок міркувань («thinking»): модель спершу розписує варіанти, перевіряє себе й лише потім видає коротку відповідь. Ці токени генеруються й рахуються в `eval_count`, хоча користувач їх не бачить (у нас 1329 токенів проти 5 за однакове «Київ»).
2. *Де міркувальні моделі виграють?* На задачах із кількома кроками, пастками й перевіркою (арифметика, логіка, код, планування); у нас 5/5 проти 0/5 на задачі про ручку й зошит. На простих фактах, форматуванні й перефразуванні вони невиправдані: дорожче й повільніше без виграшу в якості.
3. *Чим відрізняється навчання?* Інструктивна модель донавчається на парах «інструкція — відповідь» і за оцінками людей. Міркувальну додатково навчають з підкріпленням на задачах із перевірною відповіддю, заохочуючи довгі ланцюжки міркувань, що ведуть до правильного результату.
4. *Чи означає більше токенів «розумніша»?* Ні: кількість токенів — це витрати, а не якість. На простому питанні вони були зайвими; користь залежить від типу задачі.
5. *Чому різна швидкість (токенів/с)?* Розмір і архітектура моделі (4B проти 3B), довжина контексту, який росте під час довгого міркування, квантизація та навантаження GPU. У нас 62 проти 105 токенів/с; різниця в загальному часі (20.1 проти 2.4 с) переважно від кількості токенів.

Мої висновки. Міркувальну модель варто вмикати лише там, де інтуїтивна відповідь може бути хибною (задачі з пасткою), і не використовувати для простих фактів. Інструктивна мала модель швидка, але на логіці дає впевнено хибні відповіді й до того ж відповідає російською на україномовне питання. Автоперевірку відповіді (`auto_correct`) треба ретельно налаштовувати: вона не розпізнала правильні відповіді у форматі `\boxed{5}`.

Завдання 2. Роль system-повідомлення (`hello.py`)

Питання варіанта: «Поясни різницю між реляційною і документоорієнтованою базою даних.» (`temperature=0.7`). Кількість речень — оцінка скрипта за розділовими знаками (пункти списку теж рахуються як речення).

Модель	System	Довжина відповіді, символів	Мова відповіді	Кількість речень	Інструкцію виконано?
llama3.2:3b	лаконічний асистент	1944	українська, але з чужими символами й словами («□нювання», «dati»)	≈16	ні: відповідь розгорнута, а не лаконічна
llama3.2:3b	українською, 2 речення	244	українська з англійським словом «organized»	1	частково: речень не більше двох, але є англійські вкраплення

Модель	System	Довжина відповіді, символів	Мова відповіді	Кількість речень	Інструкцію виконано?
qwen3:4b	лаконічний асистент	758	українська	≈18 (список із 6 пунктів)	ні: розгорнутий структурований список
qwen3:4b	українською, 2 речення	312	українська	2	так

Модель	System	prompt_tokens	completion_tokens	total_tokens
llama3.2:3b	лаконічний асистент	60	566	626
llama3.2:3b	українською, 2 речення	64	65	129
qwen3:4b	лаконічний асистент	69	1900	1969
qwen3:4b	українською, 2 речення	78	645	723

Відповіді на питання

1. *Яка модель точніше дотрималась обмеження «максимум двома реченнями»?* З другим system-повідомленням обидві вклалися (llama — одне довге речення, qwen — два), але у qwen3:4b текст чистіший, а в llama3.2:3b є англійське «organized». З першим («лаконічний асистент») не впоралась жодна: llama видала ≈16 речень, qwen — список із 6 пунктів; формулювання «лаконічний» надто розмите.
2. *Чи були випадки, коли модель відповіла не українською?* Повністю російською чи англійською — ні, але llama3.2:3b в обох відповідях вставляла чужі слова й символи («нювання», «dati», «organized»). У qwen3:4b таких дефектів немає.
3. *Як змінився `prompt_tokens`?* Він зріс на 4 (llama: 60→64) і на 9 (qwen: 69→78), бо друге system-повідомлення довше: токени системного тексту входять у запит. Різниця між моделями — через різні токенізатори й службову розмітку шаблону чату.
4. *Чому в qwen3:4b `completion_tokens` значно більший?* Токени прихованого міркування (`thinking`) теж рахуються як згенеровані: 1900 токенів на відповідь довжиною 758 символів, 645 — на відповідь у 312 символів.

Відповіді чат-бота на 5 питань (стисло)

1. *Чому по-різному дотримуються інструкцій?* Модель не «виконує команди», а продовжує текст за найімовірнішим сценарієм, сформованим навчанням. Інструкції про довжину й мову конкурують зі звичками моделі (детальні відповіді на питання про бази даних), і слабша модель частіше виграє на користь звички.
2. *Чи має system більший пріоритет?* Формально це лише інша роль у шаблоні чату; пріоритет вчать під час донавчання, тому він не абсолютний і залежить від моделі. Тому system-повідомленню довіряють більше, ніж user, але без гарантій.
3. *Чому міркувальна модель витрачає більше токенів?* Бо приховане міркування рахується як вихідні токени, навіть коли видима відповідь коротка (у нас 645 токенів на 312 символів).
4. *Що буде при суперечливих інструкціях?* Зазвичай модель надає перевагу system, але результат нестабільний: може виконати user-інструкцію, змішати обидві або відповісти невпевнено. Тому суперечностей у промптах краще уникати.

5. *Чому малі моделі гірше дотримуються мови?* Менше параметрів і менше україномовних даних у навчанні; модель легко «зісковзує» на домінуючу мову, а в довгій відповіді помилки накопичуються (у нас чужі слова й символи в llama3.2:3b).

Мої висновки. System-повідомленням можна керувати поведінкою, але не надійно: з «лаконічним асистентом» обидві моделі видали розгорнуту відповідь, а з жорсткішою інструкцією («2 речення») впоралися. Тому інструкцію треба формулювати конкретно (кількість речень, мова), а критичні вимоги перевіряти кодом і за потреби повторювати запит або міняти модель.

Завдання 3. Локальна vs хмарна модель (`providers.py`, `compare.py`)

Питання варіанта: «Поясни різницю між реляційною і документоорієнтованою базою даних. Коротко.» (`temperature=0.7`; локально — Ollama на GPU Google Colab).

Показник	Локальна	Хмарна
Назва моделі	llama3.2:3b	openai/gpt-oss-120b (Groq)
Час відповіді, с	2.65	1.25
`completion_tokens`	158	469 (з них 27 — `reasoning_tokens`)
Довжина відповіді, символів	572	1294
Мова відповіді	українська з огріхами («ентитету», «значеньних полів», «формалі форматі»)	українська, природна; термінологія (ACID, JOIN, MongoDB Query Language) коректна
Фактичних помилок у відповіді	**2**	0

Дві помилки локальної моделі (відповідь з `lab1.log`, 09:28:34): (1) «з'єднаних з допомогою реляцій» — підміна понять: реляція в реляційній моделі — це сама таблиця (відношення), а не зв'язок/JOIN між таблицями, який модель тут неправильно називає «реляцією»; (2) документи названо просто «текстовими файлами чи об'єктами … у більш формалі форматі» — документоорієнтовані БД зберігають структуровані документи (JSON/BSON зі своєю схемою), а не довільний текст, тобто визначення фактично неточне. Хмарна відповідь таких помилок не містить (0).

Відповіді на питання

1. *Скільки рядків коду довелося змінити?* У програмі — жодного: `compare.py` однаковий для обох провайдерів. Відмінності винесено в словник `PROVIDERS` у `providers.py` — три значення: `base_url`, `api_key`, `model`.
2. *Яка відповідь повніша й точніша?* Хмарна. Вона називає транзакції ACID, JOIN, вбудовування й посилання між документами, горизонтальне масштабування, і підсумовує все коротким висновком. Локальна обмежується загальними фразами про таблиці й документи.
3. *Чи були в локальної моделі помилки або проблеми з українською?* Так: неіснуючі форми слів («ентитету», «значеньних»), кострубаті фрази, загальне розмите формулювання.
4. *Чому хмарна відповіла швидше, попри мережу?* Вона генерує майже втричі більше токенів швидше: ≈375 токенів/с (469 за 1.25 с) проти ≈60 токенів/с у локальної (158 за 2.65 с). Хмарна модель працює на спеціалізованому серверному обладнанні й завжди завантажена; мережева затримка (десятки мілісекунд) мала порівняно з цим виграшем.

Додатково. Хмарна модель проігнорувала слово «Коротко»: видала довгу структуровану відповідь із markdown (469 токенів проти 158), у кінці додавши короткий підсумок; тобто більша модель не гарантує виконання інструкції про довжину.

Відповіді чат-бота на 5 питань (стисло)

1. *Чому формат OpenAI став стандартом?* OpenAI першою масово зробила API чат-моделей, і навколо нього з'явилися бібліотеки й інструменти. Провайдери (Groq, Ollama та інші) підтримують сумісний інтерфейс, щоб користувачі переходили без переписування коду.
2. *Наскільки розмір впливає на якість?* Сильно для знань, міркування й мовної якості, але не єдиний фактор: важливі дані навчання, донавчання, квантизація, промпт і параметри. У нашому прикладі різниця велика: 3B дає розмиту відповідь з огріхами, 120B — точну.
3. *Чому хмарна швидша попри мережу?* Потужні серверні GPU, відсутність завантаження моделі (вона вже в пам'яті) й оптимізований інференс дають сотні токенів/с; мережева затримка (десятки мс) мала порівняно з виграшем.
4. *Ризики приватності?* Дані покидають вашу інфраструктуру: можливі зберігання, логування, навчання на запитах, витоки. У комерційних проєктах використовують угоди про обробку даних (DPA), режим без збереження, маскування персональних даних, приватні розгортання або локальні моделі.
5. *Що таке rate limit?* Обмеження на кількість запитів і токенів за період на безкоштовному тарифі. Його «обходять» легально: кешують відповіді, групують запити, повторюють із затримкою (backoff), чергують моделі або переходять на платний тариф.

Мої висновки. Локальну модель обираю, коли дані конфіденційні, потрібна робота без мережі або завдання просте й формальне (форматування, витяг даних із наданого тексту). Хмарну — коли потрібні знання, міркування й якісна українська, а передача даних назовні допустима. Різниця в якості між llama3.2:3b і gpt-oss-120b на нашому питанні така, що для змістовних відповідей локальна 3B-модель недостатня.

Завдання 4. Швидкість: TTFT (`benchmark.py`, `results.csv`, `chart.py` → `chart.png`)

Промпти варіанта: короткий — «Столиця Мексики?»; середній — «Поясни в одному абзаці, що таке рекурсія.»; довгий — «Напиши інструкцію з 10 пунктів, як задокументувати проєкт.» Медіана з 3 повторів після прогріву; значення беруться з `results.csv` і `lab1.log`.

Розміщення	Модель	Промпт	TTFT, с	Всього, с	Симв./с
локальна	llama3.2:3b	короткий	0.030	0.720	169.9
локальна	llama3.2:3b	середній	0.033	1.206	242.0
локальна	llama3.2:3b	довгий	0.070	6.513	291.6
хмарна	openai/gpt-oss-120b	короткий	0.274	0.367	176.4
хмарна	openai/gpt-oss-120b	середній	0.314	0.585	694.5
хмарна	openai/gpt-oss-120b	довгий	0.329	2.029	1203.1

Холодний і теплий старт локальної моделі. Холодний — перший запит («розігрів») одразу після `ollama stop llama3.2:3b`; теплий — медіана після прогріву на короткому промпті (модель уже в пам'яті).

Показник	Холодний старт	Теплий старт
TTFT, с	2.366	0.030
Всього, с	5.601	0.720

Відповіді на питання

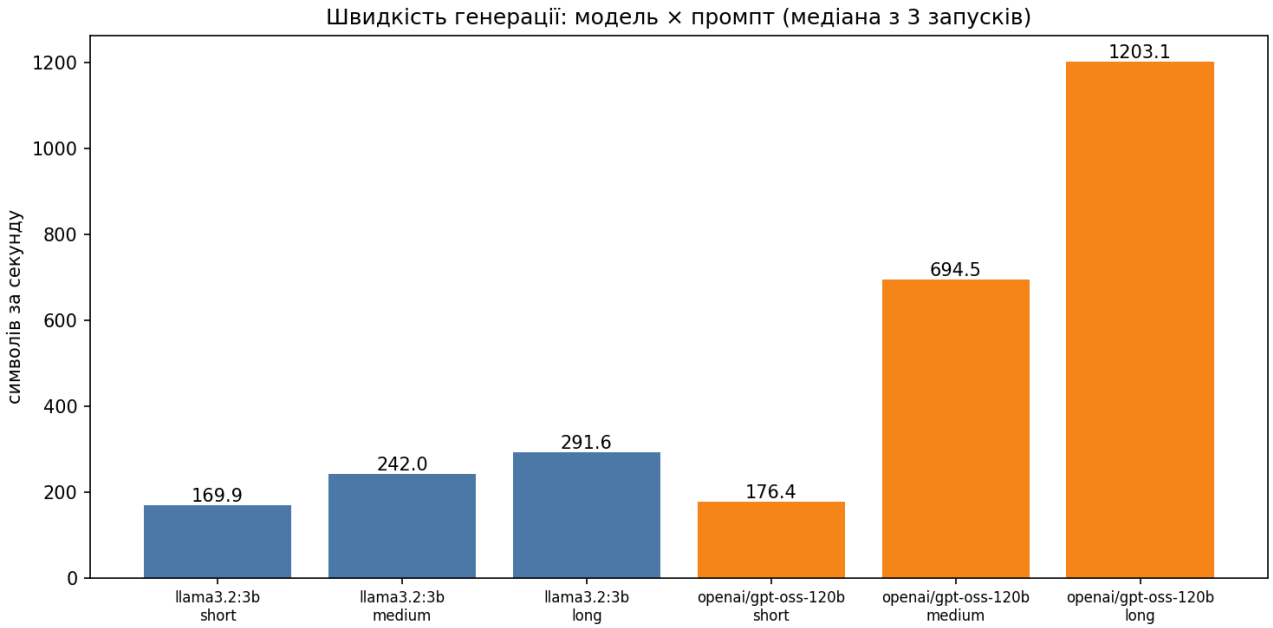


Figure 1: chart

1. *У якої моделі кращий TTFT і чому?* У локальної: 0.03–0.07 с проти 0.27–0.33 с у хмарної. Запит не йде мережею, модель мала й уже в пам'яті GPU; у хмарної в TTFT входять мережева дорога, TLS, черга на сервері й обробка запиту великою моделлю.
2. *Чи збігається «найшвидша за TTFT» з «найшвидшою за символами/с»?* Ні. За TTFT виграє локальна, а за швидкістю генерації на середньому й довгому промптах — хмарна (695 і 1203 симв./с проти 242 і 292). На короткому промпті швидкості близькі (176 проти 170).
3. *Як змінилося співвідношення при переході від короткого до довгого?* На короткому загальний час 0.37 с у хмарної проти 0.72 с у локальної (≈2×), на довгому — 2.03 проти 6.51 с (≈3.2×): чим довша відповідь, тим помітніша перевага потужного серверного обладнання.
4. *Наскільки різняться три повтори? Чи достатньо одного запуску?* TTFT повторів коливається в межах десятків відсотків (локальна довгий: 0.050–0.084 с; хмара довгий: 0.297–0.356 с; локальна короткий: 0.027–0.055 с). Одного запуску недостатньо: окремі значення зміщують результат, тому беруть медіану.
5. *Чому холодний старт повільніший?* Під час холодного старту ваги моделі читаються з диска й завантажуються в пам'ять GPU, і лише потім починається обробка запиту: TTFT 2.37 с проти 0.03 с у теплому режимі.

Відповіді чат-бота на 5 питань (стисло)

1. *Що відбувається під час TTFT?* Модель обробляє весь вхідний запит (prefill), будує кеш уваги й генерує перший токен; для хмари додаються мережа, TLS і черга. Його вимірюють окремо, бо він визначає відчуття «чат відповідає одразу», а швидкість генерації — як швидко відповідь дописується.
2. *Чому хмара може мати гірший TTFT, але кращу швидкість?* Мережева дорога й черга додають десятки-сотні мс до першого токена, а потужне обладнання потім генерує токени набагато швидше (у нас 0.33 с TTFT, але 1203 симв./с на довгому промпті проти 292 локально).
3. *Що таке холодний старт?* Перший запит після простою вимагає завантажити ваги з диска в пам'ять (GPU); у нас TTFT 2.37 с проти 0.03 с у теплому режимі.
4. *Чому медіана?* Окремі запуски дають викиди через завантаження системи, мережу,

кеші; медіана стійка до викидів, одне значення — ні.
5. *Чому вхід обробляється швидше за генерацію?* Вхідні токени обробляються паралельно одним проходом, а вихідні генеруються по одному, кожен залежить від попереднього.

Мої висновки. Для інтерактивного чат-бота важливіший TTFT (відчуття миттєвої реакції) і стабільність, для фонової обробки великих обсягів тексту — пропускна здатність (символи за секунду) і ціна. У наших вимірах локальна модель виграє за TTFT, а хмарна — за швидкістю на довгих відповідях.

Завдання 5. Якість за рубрикою (`quality.py`, `quality.json`, `check_format.py`)

Завдання варіанта 8 (тема «Туризм»):

1. Факт: рік відкриття нинішньої будівлі Одеського оперного театру. **Правильна відповідь: 1887** (відкрито 13 жовтня 1887 р.). Джерело: Вікіпедія — Одеський театр опери та балету.
2. Формат: JSON `landmark`, `city`, `year`.
3. Стиснення тексту про Одеський оперний театр до двох речень зі збереженням чисел (1810, 800, 1873, 1884, 1887, 2007; текст — у `quality.py`).
4. Код: `trip_cost(days, daily_cost, transport) = days * daily_cost + transport`.
5. Логіка: поїздка 6 днів, житло 900/добу, їжа 500/день, дорога 2400, троє, житло й дорога порівну. **Фінальна прийнята відповідь — 5600 грн**, за трактуванням «їжа 500/день — на людину»: (6·900 + 2400)/3 + 6·500 = 2600 + 3000 = 5600 грн. Саме це трактування використано в оцінках моделей нижче. Для довідки: альтернативне трактування (їжа 500/день — спільний бюджет на трьох) дає 2600 + 1000 = 3600 грн; за ним qwen3:4b (відповідь якраз 3600 грн) отримав би 2 бали замість 0 — ці альтернативні бали вказано в дужках у таблиці нижче, але в підсумку враховано основне трактування.
6. Українська мова: «Чим хостел відрізняється від готелю».

Оцінки виставлено за рубрикою з методички; для кожної моделі враховано обидва прогони (оцінки двох прогонів збіглися в усіх 18 випадках). Оцінки — мої; перевірте їх, перечитавши `quality.json`, і за потреби виправте.

Завдання	llama3.2:3b	qwen3:4b	openai/gpt-oss-120b
1. Факт	0	0	2
2. Формат	2	2	2
3. Стиснення	0	2	2
4. Код	2	2	2
5. Логіка	0	0 (2 при трактуванні 3600)	2
6. Українська	0	1	2
Разом /12	**4**	**7** (9 при трактуванні 3600)	**12**

Обґрунтування оцінок: - **1. Факт.** llama: «1892» і «1866» — різні й обидві хибні → 0. qwen: «1931» і «1947» — різні й обидві хибні → 0. gpt-oss: «1887», «1887» → 2. - **2. Формат.** `check_format.py`: усі 6 відповідей парсяться `json.loads()` без правок, markdown немає → 2 бали всім. (Зміст полів не оцінюється: llama видала «Одесьський оперний театр», «Odessa», роки 1929 і 1923; qwen — 1865 і 1922; gpt-oss — 1887, 1887.) - **3. Стиснення.** llama: у першому прогоні втрачено 1873 і 1884 і додано вступну фразу, у другому лишилося лише 1810 і 800 → 0. qwen: в обох прогонах усі числа збережені, 2 речення → 2. gpt-oss: усі числа збережені, 2 речення → 2 (дрібна граматична помилка «відкритою» оцінку не змінює). - **4. Код.** У всіх одна й та сама коректна функція. У gpt-oss код обгорнуто в markdown-блок «“‘python» попри «Лише код» — код працює, тому 2, але це слабкість формату. - **5. Логіка.** llama: у першому прогоні нісенітниця («гурturем», 3000, 1000, 1500), у другому перебір випадків без відповіді → 0. qwen: 3600 грн (їжа розподілена між трьома) в обох прогонах — хибно за основним трактуванням, правильно за альтернативним; хід

розв’язання послідовний. gpt-oss: 5600 грн із ходом розв’язання в обох прогонах → 2. - **6. Українська.** llama: мішанина з чужих слів («liesяє», «ubytování», «tolik»), нісенітниця (хостел названо «відрізом») → 0. qwen: два змістовні абзаци, дрібні помилки («сауні», «плавальні кімнати», «подорожники») → 1. gpt-oss: природна українська, два змістовні абзаци → 2.

Розбіжність оцінок між двома прогонами (так / ні)

Завдання	llama3.2:3b	qwen3:4b	openai/gpt-oss-120b
1. Факт	ні (обидва 0; відповіді різні: 1892 і 1866)	ні (обидва 0; відповіді різні: 1931 і 1947)	ні
2. Формат	ні	ні	ні
3. Стиснення	ні	ні	ні
4. Код	ні	ні	ні
5. Логіка	ні (обидва 0; відповіді різні)	ні (3600 і 3600)	ні
6. Українська	ні	ні	ні

Відповіді на питання

1. *Яка модель набрала найбільше балів і на яких завданнях основна різниця?* openai/gpt-oss-120b — 12 із 12; qwen3:4b — 7; llama3.2:3b — 4. Різниця у фактах (1), логіці (5) і мові (6): там, де потрібні знання, багатокрокове міркування чи гарна українська, мала модель провалюється.
2. *На яких завданнях оцінки двох прогонів розійшлися?* Оцінки не розійшлися ніде. Але самі відповіді розійшлися: на питання про рік llama назвала 1892 і 1866, qwen3:4b — 1931 і 1947. Оцінки збіглися, бо в слабких моделей провал системний, а не випадковий; розбіжність відповідей при хибності обох показує, що факт моделлю вигадується.
3. *Чи виконала хоч одна модель вимогу «без markdown» у завданні 2?* Так, усі три: усі 6 відповідей парсяться без правок. Очікуваного «обгортання в `json`» не сталося. Але вимогу «лише код» у завданні 4 `gpt-oss-120b` порушив (обгорткаpython), тобто розмір моделі не гарантує дотримання формату.
4. *Чи були проблеми з українською мовою?* Так. llama3.2:3b: чужі слова й символи, безглузді конструкції. qwen3:4b: дрібні помилки роду й лексики. gpt-oss-120b: лише дрібні огріхи.

Крок 7. Приклад провалу — завдання 1 (факт), обидві малі моделі.

- *Запит (повністю):* «Рік відкриття нинішньої будівлі Одеського оперного театру. Відповідай лише роком.»
- *Відповіді (повністю):* llama3.2:3b — «1892.» і «1866»; qwen3:4b — «1931» і «1947».
- *Правильна відповідь:* 1887 (відкриття 13 жовтня 1887 р.).
- *Чому так сталося:* це галюцинація. Модель не має бази даних, а генерує найправдоподібніший рік у такому форматі. Факт про регіональну споруду був у навчанні слабко представлений, тому модель замість «не знаю» видає впевнене число. Показово, що кожна мала модель дала два різних роки: стабільність відповіді була б ознакою знання, а розкид — ознака вигадки. Тому для фактів потрібні RAG або інструменти, а не пам’ять моделі.

Відповіді чат-бота на 5 питань (стисло)

1. *Інші підходи до оцінювання?* Стандартизовані бенчмарки (MMLU, GSM8K, HumanEval), порівняння «наосліп» людьми (арени), оцінка іншою моделлю як суддею (LLM-as-a-judge), автоматичні метрики (ROUGE, exact match), тести на безпеку та власні набори завдань для конкретного продукту.

2. *Чому ігнорують «без markdown»?* У навчанні JSON дуже часто зустрічається в блоках коду, тому модель відтворює звичну форму; інструкція лише один із сигналів. Надійніше вмикати режим structured output або чистити відповідь кодом.
3. *Чому гірше українською?* Україномовних даних у навчанні менше, ніж англомовних, токенізація менш ефективна, звідси русизми, чужі слова, помилки роду.
4. *Чому відповіді різної якості?* Вибірка токенів випадкова (temperature > 0), мікророзбіжності обчислень і стан сервера змінюють розподіл; навіть при 0 повна відтворюваність не гарантована.
5. *Що таке галюцинація?* Правдоподібна, але хибна відповідь. Модель навчена продовжувати текст правдоподібно, а не перевіряти істинність; визнання незнання рідкісне в даних і не завжди закріплене донавчанням.

Мої висновки. Рубрика достатньо надійна для грубого порівняння (4 / 7 / 12 з 12), але оцінки суб'єктивні, особливо логіка (5600 чи 3600) і мовна якість; тому оцінювали два прогони. Найпоказовіші завдання — факт (галюцинація), логіка й українська мова; формат (завдання 2) і код не розрізнили моделі. Для україномовного тексту я обрав би хмарну gpt-oss-120b; із локальних — qwen3:4b, бо вона краща за llama3.2:3b, хоча й допускає помилки.

Завдання 6. temperature і top_p (`sampling.py`)

Локальна модель `llama3.2:3b`. Творче завдання: «Придумай назву сервісу обміну речами. Відповідай лише однією назвою.» Фактичне: «Рік першого польоту людини в космос. Відповідай лише роком.» Правильна відповідь — **1961** (Ю. Гагарін, «Восток-1», 12 квітня 1961 р.). Джерело: Вікіпедія — Восток-1.

Параметр	Значення	Унікальних із 5 (творче)	Правильних із 5 (фактичне)
temperature	0.0	1	5
temperature	0.7	5	—
temperature	1.5	5	5
top_p	0.1	1	—
top_p	0.5	5	—
top_p	1.0	4	—

Творчі відповіді: при `temperature=0.0` — «Реча в обмен» ×5; при 1.5 — «Грусткий Диктор» (із зайвим текстом «Варіант сервійного імені»), «Товаришка», «Вортива», «Речевой Еволютор», «EchoTalkie»; при `top_p=0.5` — «Реча на місці», «Реча на ладьї», «Реча на лайк», «Реча на другий язык», «Реча в обмен»; при `top_p=1.0` — «Речебазар», «Речевий обмен» ×2, «Говірка», «Речевий обмін». Фактичне питання при `temperature=1.5`: «1961.», «1961», «1961.», «1961», «1961» — усі п'ять правильні, відрізняється лише оформлення.

Показник	Значення
Кількість запусків	10
temperature	0.0
Унікальних відповідей	1
Усі відповіді однакові (так / ні)	так

Відповіді на питання

1. *Температура для агента з JSON:* 0 (або близько 0.1): потрібна передбачуваність, а не різноманітність.
2. *Для генератора варіантів заголовків:* 0.7–1.0: достатньо різноманітності, але ще без безглуздих відповідей, які з'явилися при 1.5 (наприклад, відповідь не однією назвою, а з додатковим текстом).

3. *Чим top_p відрізняється за характером?* Він відсікає «хвіст» малоймовірних токенів: при 0.1 усі відповіді однакові, а вже при 0.5 з'являються п'ять різних варіантів, що зберігають спільний корінь («Реча на ...»). Тобто top_p змінює насамперед обсяг допустимих варіантів, а не їх «безглуздість».
4. *Чи означає temperature=0 повну відтворюваність?* У нашому експерименті 10 запусків дали 1 унікальну відповідь, тобто так, але це лише спостереження для цієї моделі й цього запиту. Гарантії немає (обчислення з плаваючою комою, паралелізація), у наступних лабораторних частина різниці між версіями агента може бути шумом.
5. *Найбезглуздіша відповідь при 1.5:* «Варіант сервійного імені: «Грусткий Диктор».» (порушено вимогу «лише одну назву» і вигадано слово) та «Речевой Еволютор» (російське «Речевой»). Модель вибрала токени з «хвоста» розподілу, у якому є слова інших мов і неіснуючі форми.

Додаткові спостереження. (а) Навіть при `temperature=0` «Реча в обмен» — це неграмотна назва: мала модель слабка в українській незалежно від температури. (б) Очікувалося, що при `temperature=1.5` модель вигадуватиме різні роки, але в 5 запусках усі відповіді були правильні: факт про Гагаріна модель знає «міцно», тож температура зіпсувала форму, а не зміст.

Відповіді чат-бота на 5 питань (стисло)

1. *Що робить temperature?* Ділить логіти (оцінки токенів) на T перед softmax: при T<1 розподіл «гострішає» і домінує найімовірніший токен, при T>1 вирівнюється і зростає шанс рідкісних токенів; T→0 дає жадібний вибір.
2. *Чим top_p відрізняється?* Це відсікання: з токенів, відсортованих за ймовірністю, лишають найменшу множину з сумою ≥ p і вибирають лише з неї. Temperature змінює форму розподілу, top_p обрізає хвіст.
3. *Змінювати обидва чи один?* Зазвичай один: їх ефекти перетинаються й важко розрізнити внесок кожного. Практична порада — змінювати temperature, залишаючи top_p за замовчуванням, або навпаки.
4. *Чому при T=0 відповіді можуть різнитися?* Через нестабільність обчислень з плаваючою комою на GPU, порядок паралельних операцій, пакетування запитів і оновлення моделі на стороні сервера; у нашому локальному експерименті відповіді збігалися, але гарантій немає.
5. *Типові значення?* Для витягу структурованих даних 0–0.2, для відповідей на питання 0.3–0.7, для творчого тексту 0.7–1.0 (інколи вище).

Мої висновки. Для наступних лабораторних, де модель має повертати структуровані дані (аргументи інструментів, JSON), використовую `temperature = 0`: потрібні відтворюваність і передбачуваність (у лабораторних 2 і 3 так і зроблено). Налаштування «за замовчуванням» (≈0.7–1.0) може виявитися непридатним: повторні запуски дають різні відповіді, що псує тести й ускладнює налагодження; при 1.5 модель узагалі видає слова з чужих мов.

Підсумок

Критерій	Локальна мала (3B): llama3.2:3b	Локальна більша (4B): qwen3:4b	Хмарна: openai/gpt-oss-120b
Швидкість генерації	170–292 симв./с; 105 токенів/с	≈59 токенів/с (+ приховане міркування: тисячі токенів)	176–1203 симв./с на промптах різної довжини
TTFT	0.03–0.07 с (теплий); 2.4 с (холодний)	не вимірювався (benchmark лише для llama3.2:3b)	0.27–0.33 с
Якість, балів /12	4	7	12

Критерій	Локальна мала (3B): llama3.2:3b	Локальна більша (4B): qwen3:4b	Хмарна: openai/gpt-oss-120b
Стабільність формату (завдання 2)	2/2	2/2	2/2 (але порушив «лише код» у завданні 4)
Потрібні ресурси	кілька ГБ пам’яті на диску й у RAM/VRAM, бажано GPU	те саме, трохи більше; довга генерація через міркування	лише мережа й API-ключ
Вартість	безкоштовно (своє обладнання)	безкоштовно (своє обладнання)	безкоштовний тариф з лімітами на запити
Куди йдуть ваші дані	залишаються на комп’ютері	залишаються на комп’ютері	надсилаються провайдеру (Groq)
Працює без інтернету	так	так	ні
Коли обирати саме її	конфіденційні дані, робота без мережі, прості формальні задачі (формат, витяг із наданого тексту, простий код); не для фактів і логіки	потрібна точність у логіці й задачах із пасткою, час і приватність важливі менше; не для простих фактів	потрібні знання, міркування й якісна українська, і дані можна передавати назовні